

Write a Makefile to automate the process

Let's write a Makefile to automate our debug build. This example is really inferior to the Makefile standards but useful to illustrate the concept we are discussing.

```
ifdef debug
    CFLAGS = -Wall -g -DMYDEBUG
else
    CFLAGS = -Wall
endif

all:myprogram

myprogram:main.c debug.h
    cc -o myprogram main.c $(CFLAGS)

clean:
    rm -rf myprogram
```

You can save this file as *Makefile* in the same directory where *main.c* resides, and run it using the following commands.

For debug build:

```
make myprogram debug=1
```

For production build:

```
make myprogram
```

It is clear from the above snippets that *debug=1* causes *CFLAGS* to include the option *-DMYDEBUG*, which will cause the C compiler to compile the lines that you placed under *#ifdef MYDEBUG*. You might have noticed an additional option *-g*, which is not necessary here, but will be useful while dealing with debuggers.

 **Note:** Do not forget to run *make clean* before making a production build after a debug build, and vice versa.

Why you need a debugger

You can't make use of *printf* debugging without having spotted some suspicious code already, unless you are ready to insert it in every other line. On the other hand, a debugger lets you do the following:

- Identify the exact line of code where a fatal error occurs
- Set breakpoints
- Watch the values and status of variables (including human-readable status of a structure variable) at crashes and breakpoints
- Get the stack trace, function backtracks and live CPU register status

- Make real-time edits to the program

To quote Richard Stallman's words from the GDB (GNU Debugger) manual, "The purpose of a debugger such as GDB is to allow you to see what is going on inside another program while it executes—or what another program was doing at the moment it crashed."

The most important advantage is that you don't have to add any additional code to your program in order to make it debuggable. Any program compiled with a simple debug flag can be run and analysed using a debugger.

If you find classical languages like C to be unfriendly compared to the modern ones like Python, you are going to have a rethink after using a debugger for the first time.

Why you may not need a debugger

Not all programmers like debuggers. This includes Linus Torvalds, Guido van Rossum and Brian Kernighan. Rossum is said to use print statements for 90 per cent of his debugging.

One reason not to use debuggers is the ease they provide, which helps you fix the last erroneous lines easily while leaving other non-fatal issues unnoticed. On the other hand, if you go through each and every line without the precise information that a debugger provides, there is a chance you might notice more logical errors. Even if there are no other errors, you'll end up rewriting parts for clarity, efficiency and robustness, while searching for errors.

Moreover, debuggers sometimes tend to be less productive, presenting you with such a comprehensive error report, that understanding it requires more time than writing a hundred *printf* lines.

But debuggers are not a total waste of time. They are the best choice to give you a stack trace, backtrack or live CPU register status. My personal advice is to learn them and use them like secret weapons, not like daily tools. Also, you can consider using one as your primary debugging tool for one week or so, in order to understand the usual mistakes you make, and you can be a better *printf* debugger afterwards.

Compiling with the debug flag

Adding *-g* option to the *gcc* command will cause the program to be compiled with debugging information, which is required if you want to communicate with the debugger in terms of human-readable information like the source file name, line number, identifiers and the source line itself.

Here is an example:

```
gcc -o myprogram -g main.c
```

Getting started with GDB

GDB, the GNU Debugger, is a free software portable debugger that supports multiple languages including C, C++, Go, Fortran and Java. If it doesn't come as part of your GNU/Linux distro,